Game Specification

Purpose of This Document

This document is the authoritative specification for the game.

The game must be completed and playable.

Claude Code should read the complete specification before implementation begins.

The highest priority is to produce a complete, playable and enjoyable game. Prefer the simplest implementation that satisfies this specification. Avoid unnecessary architecture, dependencies, abstractions or features.

A playable end-to-end version of the core game should be established as early as possible.

All **Must Have** requirements must be completed before any **Stretch** requirements are implemented.

Where practical, game content must be separated from game logic so that other team members can continue creating and refining content while development is underway.

The game should be run and checked regularly throughout development. Integration and testing should not be left until the end.

Where this specification is ambiguous, choose the simplest reasonable interpretation that is consistent with the rest of the document.

---

1. Game Concept

Working Title

[Enter working title]

One-Sentence Pitch

[Describe the game in one sentence.]

Player Role

[Who or what is the player?]

Player Objective

[What is the player trying to achieve?]

Core Gameplay Loop

Describe the main activity that the player repeats during the game.

> [Player encounters something] → [Player takes an action] → [Game responds] → [Next challenge / situation]

Game Length

[How long should a typical playthrough last?]

Tone

Examples:

- Funny
- Chaotic
- Serious
- Competitive
- Absurd
- Relaxing
- Tense

[Describe the intended tone.]

What Makes It Fun?

[Describe the main hook or reason somebody should want to keep playing.]

---

2. Gameplay and Rules

Player Actions

Describe everything the player can do.

Action	Input	Description

Game Rules

Describe the rules in clear, plain English.

1. [Rule]
2. [Rule]
3. [Rule]
4. [Rule]

Starting State

[What is true when a new game begins?]

Scoring / Success

[How is success measured? Include numbers or formulas where useful.]

Failure / Game Over

[Exactly what causes the game to end?]

Progression and Difficulty

[How does the game change as the player continues?]

Examples might include:

- increasing speed;
- harder decisions;
- more enemies or challenges;
- less available time;
- larger penalties;
- increasingly difficult content.

Randomness

[Describe anything that should be randomised between games.]

If randomness is not required, state:

> No randomness is required.

Restart / Replay

[Describe what happens after the game ends and how the player starts another game.]

---

3. Game Content

Game content should be kept separate from application logic wherever practical.

This section does **not** need to contain every piece of finished game content before development begins.

Once the format for one unit of content has been agreed, development can begin while team members continue producing additional content in parallel.

Content Type

Describe what constitutes one independently authored piece of game content.

Examples could include:

- a narrative event;
- a question;
- a card;
- an enemy;
- a challenge;
- a level;
- a character interaction;
- an item;
- a puzzle.

[Describe the content unit for this game.]

Content Fields

Define the information required for each content item.

Field	Description	Required?	Example
ID	Unique identifier	Yes	

Where practical, each independently authored content item should be representable as a single row in the team's shared content spreadsheet.

Initial Content

Provide enough finished content here to allow development of a playable version of the game.

[Add initial game content.]

Additional Content

Additional content may continue to be produced during development using the agreed content format.

---

4. Visual and Audio Design

Overall Visual Style

Describe the intended look and feel.

[Describe visual style.]

Examples or reference images may be pasted into this section.

Main Game Screen

Paste a wireframe, sketch or mock-up below.

[Insert main game screen design]

Label important elements where necessary.

Other Screens

Only include screens that are actually required.

Possible examples:

- Start screen
- Instructions
- Gameplay
- Pause
- Game over
- Results

[Screen Name]

Purpose:
[What is this screen for?]

Content:
[What appears on it?]

Player interactions:
[What can the player do here?]

[Insert mock-up if useful.]

UI

Describe important interface elements.

Examples:

- score;
- health;
- progress;
- timers;
- buttons;
- status indicators;
- resources.

[Describe required UI.]

Colours and Typography

[Describe the colour palette, fonts and general presentation.]

Animation and Visual Feedback

Describe important visual feedback.

Examples:

- transitions;
- button feedback;
- particles;
- screen shake;
- movement;
- score animations;
- hit effects.

[Describe required effects.]

Audio

Music

[Describe required music, or state that none is required.]

Sound Effects

Event	Sound / Style

---

5. Technical Specification

Technology Stack

The game will use:

- **Language:** TypeScript
- **Game / rendering framework:** PixiJS 8
- **Development and build tooling:** Vite
- **Package manager:** pnpm
- **Runtime game content:** JSON
- **Team content authoring:** Microsoft Excel / shared spreadsheet
- **Styling outside the PixiJS canvas:** Plain CSS where required
- **Persistence:** Browser `localStorage` only if required
- **Target:** Modern desktop web browsers

The game must be completely client-side.

Do not introduce a backend, database or runtime network service unless explicitly required elsewhere in this specification.

Do not introduce React, Vue, an ECS framework, a state-management framework or other major architectural dependency unless there is a clear requirement for it. If there is, use Svelte and keep it clean and simple.

Development Priorities

Prioritise:

1. A complete playable gameplay loop.
2. Correct implementation of the core mechanic.
3. Easy integration of team-created content.
4. Clear and responsive player feedback.
5. Visual and audio polish.
6. Stretch functionality.

Do not prioritise architectural sophistication over completing the game.

Content Architecture

Game content must be separated from game logic wherever practical.

Non-technical team members will create and edit content in a shared spreadsheet.

Where practical:

> One independently authored content item should correspond to one spreadsheet row.

The spreadsheet content should be exported as CSV and converted into validated JSON for use by the game.

The exact content schema must be based on the **Game Content** section of this specification.

Gameplay balancing values should also be kept outside the core application logic wherever practical so that they can be adjusted quickly.

Content Validation

Externally authored content must be validated before it is consumed by the game.

Validation should include all relevant checks, including:

- required fields are present;
- IDs are unique;
- values use the correct data types;
- permitted values are valid;
- numeric values fall within sensible ranges;
- references to other content IDs are valid;
- referenced assets exist;
- required text is not empty;
- malformed content cannot cause the game to crash;
- any additional game-specific constraints are satisfied.

Validation errors should clearly identify the affected content item or spreadsheet row and explain the problem.

Project Structure

Use a simple, understandable project structure.

A suitable starting structure is:

```
src/
  main.ts
  game/
  scenes/
  systems/
  ui/
  types/
  utils/

public/
  assets/
    images/
    audio/
  data/

content/
  game-content.csv

scripts/
  build-content.ts
```

Adapt this where necessary for the chosen game.

Build and Quality Checks

Provide the following commands:

`pnpm run dev`

Runs the game locally for development.

`pnpm run build`

Creates the production build.

`pnpm run check`

Runs the project's automated quality checks.

`pnpm run check` should, at minimum:

1. validate game content;
2. run TypeScript type checking;
3. verify that the production build succeeds.

A broad automated unit-test suite is **not required** unless the chosen game contains complex standalone logic that clearly benefits from automated tests.

Gameplay, visuals, controls and overall experience should primarily be verified through manual play-testing.

6. Scope and Definition of Done

Must Have

These requirements define the minimum complete game.

The game is not finished until every Must Have requirement works.

- ☐ [Requirement]
- ☐ [Requirement]
- ☐ [Requirement]
- ☐ [Requirement]
- ☐ [Requirement]

Stretch

Only implement these after all Must Have functionality is complete and the full game has been successfully played from beginning to end.

- ☐ [Stretch feature]
- ☐ [Stretch feature]
- ☐ [Stretch feature]

Definition of Done

The game is complete when:

- ☐ It launches successfully.
- ☐ A new player can understand how to play.
- ☐ The complete core gameplay loop works.
- ☐ Win, success, failure or game-over behaviour works as specified.
- ☐ The player can complete a full playthrough.
- ☐ The player can restart or play again without manually refreshing or restarting the application.
- ☐ Team-created content loads correctly.
- ☐ Invalid content is detected by the validation process.
- ☐ `npm run check` completes successfully.
- ☐ There are no known errors that prevent normal gameplay.
- ☐ The game has been play-tested by at least one team member other than the developer.
- ☐ All Must Have requirements are complete.

Final Play-Test

Before development is considered finished, play through the game as a new player would:

> Launch → Understand what to do → Start → Play the complete gameplay loop → Reach the ending / game over → Play again

Fix issues that prevent or significantly harm this experience before implementing additional Stretch functionality.